Hash /Map

Jianguo Lu

February 23, 2025

Overview

Lab Assignment 4: Implement Hash and Heap ADTs

Fast frequency counting

```
public static AbstractMap.SimpleEntry<String, Integer> countHash(String[] tokens){
  HashMap2540<String, Integer> map = new HashMap2540<String, Integer>();
  int len = tokens.length;
  for (int i = 0; i < len; i++) {
        String token = tokens[i];
        Integer freq = map.get(token);
        if (freq == null)
                map.put(token, 1);
        else
                map.put(token, freq + 1);
  }

  int max = 0;
  String maxWord="";
  for (String k : map.keys())
        if (map.get(k) > max){
                max = map.get(k);
                maxWord=k;
        }
  return new AbstractMap.SimpleEntry<String, Integer>(maxWord, max);
}
```

A4: HashMap2540

```
public class HashMap2540<Key, Value> {
        private int n; // number of elements
        private int m; // hash table size
        private LinkedListForHash2540<Key, Value>[] st;  // array of linked-list.

        // hash value between 0 and m-1
        private int myhash(Key key) {
                int hash = 7;
                String k = (String) key; //here we assume keys are strings.
                int base=31;
                for (int i = 0; i < k.length(); i++)
                        //calculate hashcode (h1) using polynomial method:
                        // hash=base^{n-1} key[0]+base^{n-2} key[1] +....+key[n-1]
                        hash=(int)Math.round((Math.pow(31,k.length()-i-1)))*k.charAt(i)+hash;
                //Implement h2 using division method
                hash=Math.abs(hash) % m;
                return hash;
        }

        public Value get(Key key) {
                int i = myhash(key);
                return st[i].get(key);
        }

        public void put(Key key, Value val) {
                // 1) get the hash value of the key i.
                // 2) then put (key, value) in the i-th linkedList implemented in LinkedListForHash254
                // 3) you need to handle the case whether the key is already there.
        }

        public void delete(Key key) {

                int i = myhash(key);
                if (st[i].get(key)!=null)
                        n--;
                st[i].delete(key);

        }
}
```

LinkedListForHash2540

```
import java.util.LinkedList;
public class LinkedListForHash2540<Key, Value> {
    private int n;           // number of key-value pairs
    private Node first;      // the linked list of key-value pairs

    public Value get(Key key) {
        for (Node x = first; x != null; x = x.next) {
            if (key.equals(x.key))
                return x.val;
        }
        return null;
    }

    public void put(Key key, Value val) {
        if (val == null) {
            delete(key);
            return;
        }

        for (Node x = first; x != null; x = x.next) {
            if (key.equals(x.key)) {
                x.val = val;
                return;
            }
        }
        first = new Node(key, val, first);
        n++;
    }

    public void delete(Key key) {
        first = delete(first, key);
    }

    private Node delete(Node x, Key key) {
        if (x == null) return null;
        if (key.equals(x.key)) {
            n--;
            return x.next;
        }
        x.next = delete(x.next, key);
        return x;
    }
}
```

Dynamic array (size of the hash table)

```
public class HashMap2540<Key, Value> {
        private static final int INIT_CAPACITY = 4;
        private int n; // number of elements
        private int m; // hash table size
        private LinkedListForHash2540<Key, Value>[] st;
        // array of linked-list.

        public HashMap2540() {
                this(INIT_CAPACITY);
        }

        // used in resize.
        public HashMap2540(int m) {
                this.m = m;
                st = (LinkedListForHash2540<Key, Value>[]) new LinkedListForHash2540[m];
                for (int i = 0; i < m; i++)
                        st[i] = new LinkedListForHash2540<Key, Value>();
        }

        // resize the hash table to have the given number of chains,
        // rehashing all of the keys
        private void resize(int n) {
                HashMap2540<Key, Value> temp = new HashMap2540<Key, Value>(n);
                for (int i = 0; i < m; i++) {
                        for (Key key : st[i].keys()) {
                                temp.put(key, st[i].get(key));
                        }
                }
                this.m = temp.m;
                this.n = temp.n;
                this.st = temp.st;
        }


        public void put(Key key, Value val) {
                // double table size if average linked list size is 10.
                if (n >= 10 * m)
                        resize(2 * m);
                // Put your code here.
                // 1) get the hash value of the key i.
                // 2) then put (key, value) in the i-th linkedList implemented in LinkedListForHash254
                // 3) you need to handle the case whether the key is already there.
        }
```

Heap2540

```
public class Heap2540 {
    private String[] heap;
    private int n=0;

    public Heap2540(String[ ] a) {
     // define the constructor for a heap from an array
    }

    public Heap2540() {
        heap=new String[128];
    }

    public String removeMax() {
        String max=heap[1];
            swap(1, n);
        n--;
        sink(1);
        return max;
    }

    public void insert(String x) {
        // add resize
        n++;
        heap[n] = x;
        swim(n);
    }

    private void swim(int k) {
     // add swim
    }

    private void swap(int i, int j) {
        String temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    private void sink(int k) {
     // add definition here
    }

    private boolean less(int i, int j) {
        return heap[i].compareTo(heap[j]) < 0;
    }
```

Map/Dictionary

- Map is a set of (key, value) pairs
- Examples
  - (word, def): look up word in a dictionary
  - (word, freq): Frequency count
  - (word, webPages): Get all the web pages containing the word. Google search engine.
  - (StudentID, name)
  - (SIN, name)
  - (PhoneNumber, address)
  - (url, webPage):
  - (hostName, IP address): e.g.,(uwindsor.ca, 162.219.54.2)
  - (varName, datatype) in compiler
  - ...

Map ADT

Key operations in a Map ADT

get(k) : Returns the value associated with key k, if such an entry exists;

put(k,v) : If k exists, replace its value with v; otherwise add entry (k,v);

remove(k) : Remove the entry with key equal to k,

- A bad Implementation: using LinkedList
- Time complexity: $O(n)$

Motivating example: word count

Count the frequency of each word in document(s)

```
public static void main(String[] args)
  Map<String,Integer> freq = new ChainHashMap<>();
  Scanner doc = new Scanner(System.in).useDelimiter("[^a-zA-Z]+");
  while (doc.hasNext()) {
    String word = doc.next().toLowerCase();
    Integer count = freq.get(word);
    if (count == null) count = 0;
    freq.put(word, 1 + count);
```

- Crucial operations we need?
- *get*(*word*) and *put*(*word*, *freq*) methods
- *word* is called key, *freq* is its value

Map ADT in Java

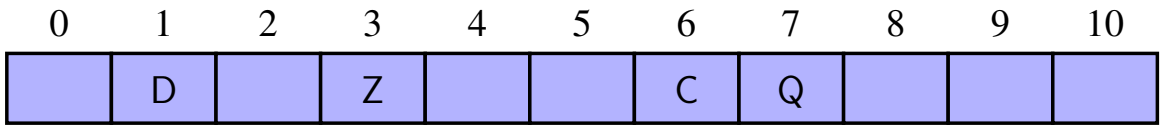
```
public interface Map<K,V> {
  V get(K key);
  V put(K key, V value);
  V remove(K key);

  int size();
  boolean isEmpty();
  Iterable<K> keySet();
  Iterable<V> values();
  Iterable<Entry<K,V>> entrySet();
}
```

Hash (etemology)

hash1
/haSH/Submit
noun
noun: hash; plural noun: hashes
1. a dish of cooked meat cut into small pieces and cooked again, usually with potatoes.
NORTH AMERICAN
a finely chopped mixture.
"a hash of raw tomatoes, chili peppers, and cilantro"
a mixture of jumbled incongruous things; a mess.
synonyms: mixture, assortment, variety, array, mix, miscellany, selection, medley, mishmash, ragbag, gallimaufry, potpourri, hodgepodge
"a whole hash of excuses"

Implement using an array

- Can we *get* and *put* the key in (almost) a constant time?
- Recall that array elements can be accessed in a constant time (only if we have the index).
- Need to use key as an index of an array.
- When keys are small integers, we can use keys directly as indexes.
  - E.g., when the map items are: (1,D), (3,Z), (6,C), and (7,Q). The map can implemented as an array:



- There are two problems:
  - Keys can be very large, e.g., SIN (Social Insurance Number)
  - Keys may not be non-negative integers (e.g. Strings)

Example: SIN (social insurance number)

Design a hash table for a map storing entries as (SIN, Name),

- SIN: nine-digit positive integer.
- The array can not have the length 10^9.
- The array should have the length as needed.
- If Hash table uses an array of size N=10000, we need to map SIN into a value with in 0..N-1
- the hash function can be

$$h(x) = \textit{last four digits of } x$$

- Problem:
  - All SINs with the same trailing digits will have the same hashcode
  - When two SINs have the same hashcode, we call it a collision.
  - We cannot use one array cell to store two elements (values)
- Two questions:
  - How can we handle the collision?
  - How to reduce the collisions?

Hashing and compression

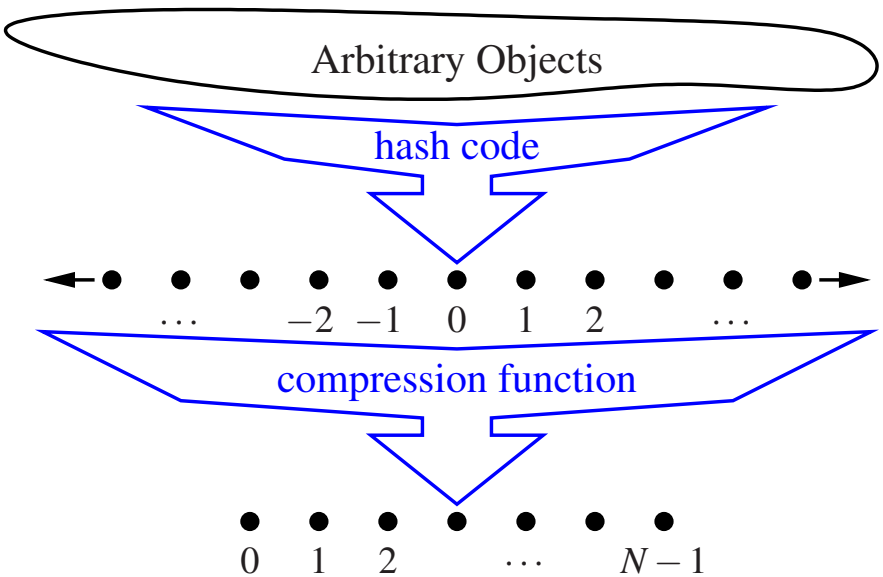
- A hash function is the composition of two functions: $h(x) = h2(h1(x))$
- Hash (prehash) function:

$$h1 : keys \rightarrow integers \tag{1}$$

- Compression function:

$$h2 : integers \rightarrow [0, N-1] \tag{2}$$

- The goal of the hash function is to "disperse" the keys in an apparently random way



Hash function f1: turn an object into an integer

Memory address: reinterpret the memory address of the key object as an integer (default hash code of all Java objects)

- Good in general, except for numeric and string keys

Integer cast: reinterpret the bits of the key as an integer.

- Suitable for keys of length less than or equal to the number of bits of the integer type (e.g., byte, short, int in Java)

Hash function: Component sum

- Partition the bits of the key into components of fixed length (e.g., 16 or 32 bits)
- Sum the components (ignoring overflows)
- Suitable for numeric keys of fixed length greater than or equal to the number of bits of the integer type (e.g., long and double in Java)

Polynomial hash code

- Given a string $x_0 x_1 \ldots x_{n-2} x_{n-1}$.
- $\sum_{i=0}^{n-1} x_i$ is not a good hash code.
- Needs to reflect the position of each character:

$$x_0 a^{n-1} + x_1 a^{n-2} + \cdots + x_{n-2} a + x_{n-1}. \tag{3}$$

- Direct calculation of the above eq is costly.
  - how many multiplication operations are performed?
- The Horner's method transforms the above to

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_2 + a(x_1 + a x_0)) \ldots)). \tag{4}$$

- Very few collisions (50,000 English words, 7 collisions)

```
int hash = 0;
for (int i = 0; i < s.length(); i++)
    hash = (a * hash + s.charAt(i)) % M;  //a is 31 for Java
```

Division method

- Compression (h2) function: A hash function $h2$ mapping integers to a fixed interval [0, N - 1]
- An simple implementation is mod function:

$$h2(x) = x \ mod \ N \tag{5}$$

- Example:

x	200	205	210	215	220	600
mod 100	0	5	10	15	20	0

- Better to use a prime number as N to reduce collisions

x	200	205	210	215	220	600
mod 101	99	3	8	13	18	95

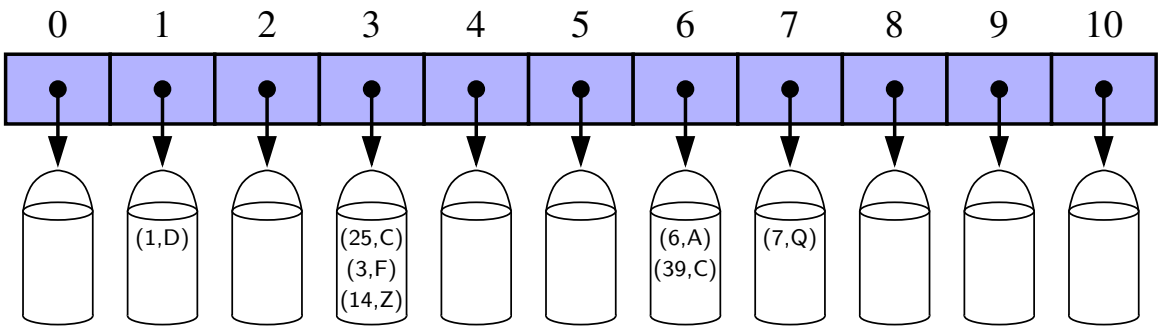
Collision problem

- Storing the following keys into a array with length 11. Suppose the compression function is:

$$hashcode(i) = i \mod 11 \tag{6}$$

key	1	3	6	7	14	25	39
hashcode	1	3	6	7	3	3	6

- One index may have multiple keys



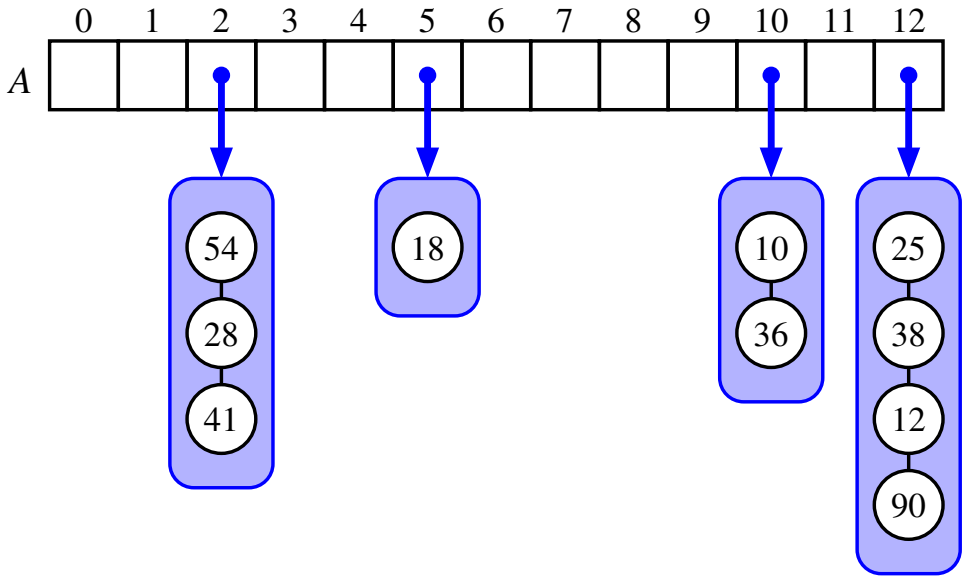
Collision

For two different keys k1 and k2, a collision happens if

$$h(k1) = h(k2)$$

Separate chaining

- A[j] stores a container to all keys k such that $h(k) = j$
- Drawback: not space efficient. requires the use of an auxiliary data structure



- Load factor: $\lambda = n/N$. (N: size of the hash table. n: number of elements)
- $\lambda = 10/13$ in this example
- λ is smaller than one.

Resizing and rehashing

- Goal: Keep λ within a range.
- Double the size of array N when λ is too large
- Halve the size of array Ne when λ is too small
- Need to rehash all keys when resizing.
- h_1 does not change but h_2 is different

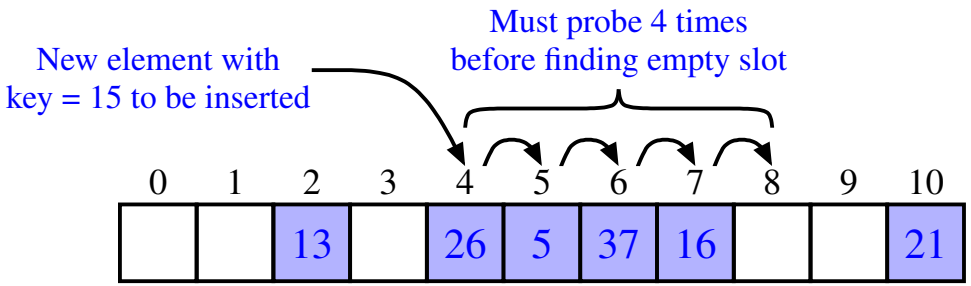
Linear Probing

- Suppose the compression function is

$$i \mod 11$$

- array length is 11
- sequence of insertion is 13, 26, 5, 37, 16, 21,15
- note that array cells store the actual values corresponding to the keys

key	hashcode	action
13	2	a[2]=(13, the value)
26	4	a[4]=(26, the value)
5	5	a[5]=(5, the value)
37	4	a[4] used, check a[4+1], check a[5+1], a[6]=(37, the value)
16	5	a[5], a[6] used, a[7]=(16, the value)
...		

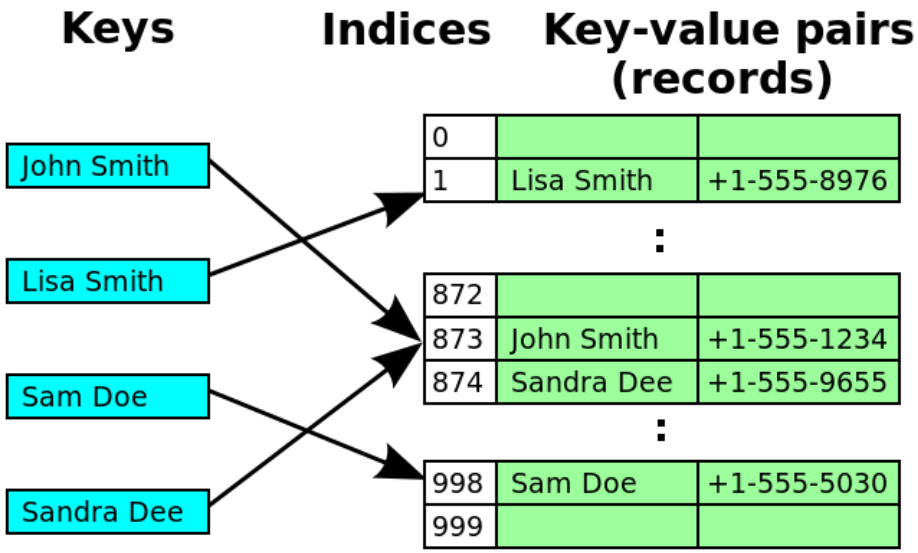


Linear probing tracing

put(w1, f1), ..., put(w7,f7)
get(w2), get(w7)
del(w5)
get(w7)

Linear Probing: a more complete picture

- note that array cells store the actual values corresponding to the keys



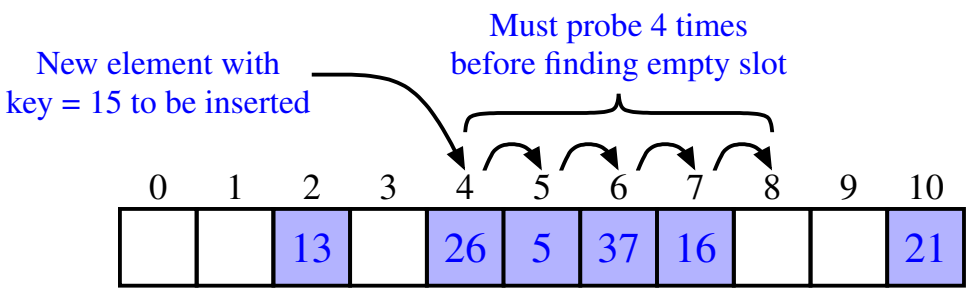
Linear Probing: get and del

- Get(k)

key	hashcode	action
26	4	a[4]= 26, get the corresponding value, done.
37	4	a[4]=37? a[5]=37? a[6]=37? done
15	4	a[5]=15? a[5]=15? ....a[8]=null. not found
...		

- remove(k)

key	hashcode	action
remove(13)	2	a[2]= null
remove(37)	4	a[4]=37? a[5]=37? a[6]=37? a[6]=sentinel
get(16)	5	a[5]=16? a[6]=sentinel continue, a[7]=16
...		



Quadratic probing and double hashing

- Linear probing may cause clustered blocks. Makes probing longer
- Solution 1: quadratic probing: placing an item in the first available cell of the series

$$\left(h(k) + i^2\right) \mod N \tag{7}$$

  - When there is a collision on h(k), try

$$\begin{aligned} &h(k) + 1, \\ &h(k) + 4, \\ &h(k) + 9, \\ &h(k) + 16, \\ &\ldots \end{aligned} \tag{8}$$

- Solution 2: Double hashing: uses a secondary hash function $h'(k)$ and handles collisions by placing an item in the first available cell of the series

$$\left(h(k) + i \times h'(k)\right) \mod N$$

It is all about space-time tradeoff

- No space limitation: trivial hash function with key as index.
- No time limitation: trivial collision resolution with sequential search.
- Space and time limitations: hashing (the real world).

Takeaways

- Why hashing?
- Hashcode, pre-hash (h1)
- Compression methods (h2)
- How do deal with collisions? Chaining, open address. load factor
- How hash table grows using dynamic array
- Readings: Goodrich et al., Pages 402-427.